

Practical 1

Ping

```
ping google.com

student@ubuntu:~$ ping -c 4 google.com
PING google.com (142.250.72.46) 56(84) bytes of data.
64 bytes from bom12s02-in-f14.1e100.net (142.250.72.46): icmp_seq=1 ttl=118
time=16.8 ms
64 bytes from bom12s02-in-f14.1e100.net (142.250.72.46): icmp_seq=2 ttl=118
time=16.6 ms
64 bytes from bom12s02-in-f14.1e100.net (142.250.72.46): icmp_seq=3 ttl=118
time=16.4 ms
64 bytes from bom12s02-in-f14.1e100.net (142.250.72.46): icmp_seq=4 ttl=118
time=16.6 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 16.434/16.611/16.837/0.151 ms
student@ubuntu:~$
```

ipconfig

```
ipconfig (ip addr)

student@ubuntu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
state UP group default qlen 1000
    link/ether 08:00:27:86:3f:2a brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.25/24 brd 192.168.1.255 scope global dynamic
noprofixroute enp0s3
        valid_lft 86315sec preferred_lft 86315sec
    inet6 fe80::a00:27ff:fe86:3f2a/64 scope link
        valid_lft forever preferred_lft forever
student@ubuntu:~$
```

Ifconfig

```
student@ubuntu:~$ ifconfig
enp8s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.25 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::a00:27ff:fe86:3f2a prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:86:3f:2a txqueuelen 1000 (Ethernet)
    RX packets 28752 bytes 38597291 (36.8 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 18456 bytes 1921014 (1.8 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 808 bytes 70925 (69.2 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 808 bytes 70925 (69.2 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
student@ubuntu:~$
```

Tracert

```
student@ubuntu:~$ traceroute google.com
traceroute to google.com (142.250.72.46), 30 hops max, 60 byte packets
 1 192.168.1.1 0.527 ms 0.476 ms 0.462 ms
 2 10.0.0.1 3.148 ms 3.096 ms 3.061 ms
 3 103.21.244.1 6.213 ms 6.180 ms 6.145 ms
 4 103.21.244.17 6.612 ms 6.578 ms 6.545 ms
 5 104.16.12.33 7.271 ms 7.239 ms 7.205 ms
 6 142.251.66.173 15.123 ms 15.089 ms 15.057 ms
 7 142.251.72.46 16.606 ms 16.572 ms 16.538 ms
student@ubuntu:~$
```

Arp -a

```
arp -a
student@ubuntu:~$ arp -a
? (192.168.1.1)      at 00:50:56:c0:00:08 [ether]  on enp0s3
? (192.168.1.24)    at 08:00:27:6b:5d:11 [ether]  on enp0s3
? (192.168.1.1)      at 08:00:27:57:13:9a [ether]  on enp0s3
? (192.168.1.255)   at ff:ff:ff:ff:ff:ff [ether]  on enp0s3
student@ubuntu:~$
```

Netstat

```
netstat -tuln
student@ubuntu:~$ netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.1:53            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp6       0      0 :::53                   :::*                   LISTEN
tcp6       0      0 :::22                   :::*                   LISTEN
student@ubuntu:~$
```

Whois

```
whois google.com

student@ubuntu:~$ whois google.com
Domain Name: GOOGLE.COM
Registry Domain ID: 2138514_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Updated Date: 2024-04-15T07:26:46Z
Creation Date: 1997-09-15T04:00:00Z
Registry Expiry Date: 2028-09-14T04:00:00Z
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.2083895740
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2024-05-25T09:42:18Z <<<

student@ubuntu:~$
```

Practical 3

```
Activities Zenmap May 25 3:15 PM

Zenmap
Scan Tools Profile Help
Target: 192.168.1.25 Profile: Intense scan Scan Cancel
Command: nmap -T4 -A -v 192.168.1.25

Hosts Services Nmap Output Ports/Hosts Topology Host Details Scans
OS Host
192.168.1.25
Starting Nmap 7.94 ( https://nmap.org ) at 2024-05-25 15:14 IST
NSE: Loaded 155 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 15:14
Completed NSE at 15:14, 0.00s elapsed
Initiating SYN Stealth Scan at 15:14
Completed SYN Stealth Scan at 15:14, 2.73s elapsed (1000 total ports)
Scanning 192.168.1.25 [1000 ports]
Discovered open port 22/tcp on 192.168.1.25
Discovered open port 80/tcp on 192.168.1.25
Discovered open port 139/tcp on 192.168.1.25
Discovered open port 445/tcp on 192.168.1.25
Discovered open port 3306/tcp on 192.168.1.25
Completed SYN Stealth Scan at 15:14, 2.73s elapsed (1000 total ports)
Initiating Service scan at 15:14
Completed Service scan at 15:14, 6.38s elapsed (5 services on 1 host)
Initiating OS detection (try #1) against 192.168.1.25
NSE: Script scanning 192.168.1.25.
Initiating NSE at 15:14
Completed NSE at 15:14, 3.21s elapsed
Nmap scan report for 192.168.1.25
Host is up (0.00046s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http      Apache/2.4.41 ((Ubuntu))
139/tcp   open  netbios-ssn Samba smbd 4.11.6-Ubuntu (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn Samba smbd 4.11.6-Ubuntu (workgroup: WORKGROUP)
1306/tcp  open  mysql     MySQL 5.7.31-0ubuntu0.18.04.1
MAC Address: 08:0C:29:6A:3B:7E (VMware)
Device type: general purpose
Running: Linux 4.X[5.X]
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.8
Network Distance: 1 hop
Service Info: Host: 192.168.1.25; OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
Hop RTT Address
1 0.46 ms 192.168.1.25

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.70 seconds
```

Practical 4

Burp Proxy

The screenshot displays the Burp Suite Community Edition v2024.4.5 interface. The top menu bar includes options like Burp, Project, Intruder, Repeater, View, and Help. The main window is divided into several panes:

- Left Sidebar:** Contains navigation options such as Home, Instructions, Setup, Brute Force, Command Injection, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored, **XSS DOM** (highlighted), CSRF Bypass, JavaScript, DVWA Security, PHP Info, About, and Logout. It also shows the current user as 'admin', security level as 'low', and PHPIDS as 'disabled'.
- Main Window:** Displays the DVWA application interface. The title is 'Vulnerability: DOM Based Cross Site Scripting (XSS)'. The form asks 'What's your name?' and has a 'Submit' button. Below the form, there is a message: 'Hello <script>alert('XSS')</script>'. Under 'More Information', there are links to various security resources.
- Bottom Pane:** Shows the intercepted HTTP request and response. The request is a GET request to 'http://localhost:80'. The response is an HTTP/1.1 200 OK from Apache/2.4.58 (Ubuntu). The response body contains HTML code, including a title 'Vulnerability: DOM Based Cross Site Scripting (XSS)' and a link to a stylesheet.

Start coding or [generate](#) with AI.

✓ Practical 5

```
def caesar_cipher(text, shift, mode='encrypt'):
    result = ""
    for char in text:
        if 'a' <= char <= 'z':
            start = ord('a')
        elif 'A' <= char <= 'Z':
            start = ord('A')
        else:
            result += char
            continue

        if mode == 'encrypt':
            shifted_char = chr((ord(char) - start + shift) % 26 + start)
        elif mode == 'decrypt':
            shifted_char = chr((ord(char) - start - shift + 26) % 26 + start)
        else:
            raise ValueError("Mode must be 'encrypt' or 'decrypt'")
        result += shifted_char
    return result
```

```
message = "Hello World"
shift_value = 3

encrypted_message = caesar_cipher(message, shift_value, mode='encrypt')
print(f"Original: {message}")
print(f"Encrypted: {encrypted_message}")

decrypted_message = caesar_cipher(encrypted_message, shift_value, mode='decrypt')
print(f"Decrypted: {decrypted_message}")
```

Original: Hello World
Encrypted: Koor Zruog
Decrypted: Hello World

✓ Practical 6

```
def monoalphabetic_cipher(text, key, mode='encrypt'):
    result = ""
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    key_map = dict(zip(alphabet, key)) if mode == 'encrypt' else dict(zip(key, alphabet))

    for char in text.upper():
        if char in alphabet:
            result += key_map[char]
        else:
            result += char
    return result
```



```

def vigenere_cipher(text, key, mode='encrypt'):
    result = ""
    key = key.upper()
    key_len = len(key)
    key_index = 0

    for char in text.upper():
        if 'A' <= char <= 'Z':
            key_char = key[key_index % key_len]
            key_shift = ord(key_char) - ord('A')

            if mode == 'encrypt':
                shifted_char = chr(((ord(char) - ord('A') + key_shift) % 26) + ord('A'))
            elif mode == 'decrypt':
                shifted_char = chr(((ord(char) - ord('A') - key_shift + 26) % 26) + ord('A'))
            else:
                raise ValueError("Mode must be 'encrypt' or 'decrypt'")
            result += shifted_char
            key_index += 1
        else:
            result += char
    return result

print("Monoalphabetic Cipher:")
mono_key = "ZYXWVUTSRQPONMLKJIHGFEDCBA"
mono_message = "HELLO WORLD"
encrypted_mono = monoalphabetic_cipher(mono_message, mono_key, 'encrypt')
print(f"Original: {mono_message}")
print(f"Encrypted: {encrypted_mono}")
decrypted_mono = monoalphabetic_cipher(encrypted_mono, mono_key, 'decrypt')
print(f"Decrypted: {decrypted_mono}")

print("\nPolyalphabetic (Vigenere) Cipher:")
vigenere_key = "KEY"
vigenere_message = "ATTACKATDAWN"
encrypted_vigenere = vigenere_cipher(vigenere_message, vigenere_key, 'encrypt')
print(f"Original: {vigenere_message}")
print(f"Encrypted: {encrypted_vigenere}")
decrypted_vigenere = vigenere_cipher(encrypted_vigenere, vigenere_key, 'decrypt')
print(f"Decrypted: {decrypted_vigenere}")

```

Monoalphabetic Cipher:
 Original: HELLO WORLD
 Encrypted: SVOOL DLIOW
 Decrypted: HELLO WORLD

Polyalphabetic (Vigenere) Cipher:
 Original: ATTACKATDAWN
 Encrypted: KXRKGIXBKAL
 Decrypted: ATTACKATDAWN

✓ Practical 7

```

def generate_playfair_key_square(key):
    key = key.upper().replace('J', 'I')
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    key_square = []
    for char in key:

```

```

        if char not in key_square and char in alphabet:
            key_square.append(char)
    for char in alphabet:
        if char not in key_square:
            key_square.append(char)

    matrix = []
    for i in range(5):
        matrix.append(key_square[i*5:(i+1)*5])
    return matrix

def find_char_position(matrix, char):
    for r, row in enumerate(matrix):
        for c, val in enumerate(row):
            if val == char:
                return r, c
    return -1, -1

def playfair_encrypt_digraph(matrix, digraph):
    r1, c1 = find_char_position(matrix, digraph[0])
    r2, c2 = find_char_position(matrix, digraph[1])

    if r1 == r2:
        return matrix[r1][(c1 + 1) % 5] + matrix[r2][(c2 + 1) % 5]
    elif c1 == c2:
        return matrix[(r1 + 1) % 5][c1] + matrix[(r2 + 1) % 5][c2]
    else:
        return matrix[r1][c2] + matrix[r2][c1]

def playfair_decrypt_digraph(matrix, digraph):
    r1, c1 = find_char_position(matrix, digraph[0])
    r2, c2 = find_char_position(matrix, digraph[1])

    if r1 == r2:
        return matrix[r1][(c1 - 1 + 5) % 5] + matrix[r2][(c2 - 1 + 5) % 5]
    elif c1 == c2:
        return matrix[(r1 - 1 + 5) % 5][c1] + matrix[(r2 - 1 + 5) % 5][c2]
    else:
        return matrix[r1][c2] + matrix[r2][c1]

def playfair_cipher(text, key, mode='encrypt'):
    matrix = generate_playfair_key_square(key)
    text = text.upper().replace('J', 'I').replace(' ', '')

    processed_text = []
    i = 0
    while i < len(text):
        char1 = text[i]
        if i + 1 == len(text):
            processed_text.append(char1 + 'X')
            i += 1
        else:
            char2 = text[i+1]
            if char1 == char2:
                processed_text.append(char1 + 'X')
                i += 1
            else:
                processed_text.append(char1 + char2)
                i += 2

```



```

result = ""
for digraph in processed_text:
    if mode == 'encrypt':
        result += playfair_encrypt_digraph(matrix, digraph)
    elif mode == 'decrypt':
        result += playfair_decrypt_digraph(matrix, digraph)
    else:
        raise ValueError("Mode must be 'encrypt' or 'decrypt'")
return result

playfair_key = "MONARCHY"
playfair_message = "INSTRUMENTS"

encrypted_playfair = playfair_cipher(playfair_message, playfair_key, 'encrypt')
print(f"Original Playfair: {playfair_message}")
print(f"Encrypted Playfair: {encrypted_playfair}")
decrypted_playfair = playfair_cipher(encrypted_playfair, playfair_key, 'decrypt')
print(f"Decrypted Playfair: {decrypted_playfair}")

```

```

Original Playfair: INSTRUMENTS
Encrypted Playfair: GATLMZCLRQXA
Decrypted Playfair: INSTRUMENTSX

```

✓ Practical 8

```

import numpy as np

def mod_inverse(a, m):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def matrix_mod_inverse_2x2(matrix, m):
    det = int(round(np.linalg.det(matrix)))
    det_inv = mod_inverse(det % m, m)
    if det_inv is None:
        raise ValueError("Matrix is not invertible modulo %d" % m)

    adjugate = np.array([[matrix[1, 1], -matrix[0, 1]], [-matrix[1, 0], matrix[0, 0]])
    inv_matrix = (det_inv * adjugate) % m
    return inv_matrix

def hill_cipher(text, key_matrix, mode='encrypt'):
    n = key_matrix.shape[0]
    text = text.upper().replace(' ', '')

    if len(text) % n != 0:
        text += 'X' * (n - (len(text) % n))

    result = []

    current_matrix = key_matrix
    if mode == 'decrypt':
        current_matrix = matrix_mod_inverse_2x2(key_matrix, 26)

```

```

for i in range(0, len(text), n):
    block = text[i : i + n]
    numeric_block = np.array([ord(char) - ord('A') for char in block]).reshape(n, 1)

    transformed_block = np.dot(current_matrix, numeric_block) % 26

    for val in transformed_block:
        result.append(chr(val[0] + ord('A')))

return "".join(result)

hill_key = np.array([[3, 3], [2, 5]])
hill_message = "ACT"

print("Hill Cipher:")
print(f"Original Hill: {hill_message}")
encrypted_hill = hill_cipher(hill_message, hill_key, 'encrypt')
print(f"Encrypted Hill: {encrypted_hill}")
decrypted_hill = hill_cipher(encrypted_hill, hill_key, 'decrypt')
print(f"Decrypted Hill: {decrypted_hill}")

```

Hill Cipher:
Original Hill: ACT
Encrypted Hill: GKWX
Decrypted Hill: ACTX

✓ Practical 9

```

def rail_fence_encrypt(text, rails):
    fence = [['\n'] * len(text) for _ in range(rails)]

    direction = -1
    row, col = 0, 0

    for char in text:
        if row == 0 or row == rails - 1:
            direction *= -1
        fence[row][col] = char
        col += 1
        row += direction

    result = []
    for i in range(rails):
        for j in range(len(text)):
            if fence[i][j] != '\n':
                result.append(fence[i][j])
    return "".join(result)

def rail_fence_decrypt(cipher, rails):
    fence = [['\n'] * len(cipher) for _ in range(rails)]

    direction = -1
    row, col = 0, 0

    for i in range(len(cipher)):
        if row == 0 or row == rails - 1:
            direction *= -1

```

```

        fence[row][col] = '*'
        col += 1
        row += direction

    index = 0
    for i in range(rails):
        for j in range(len(cipher)):
            if fence[i][j] == '*':
                fence[i][j] = cipher[index]
                index += 1

    result = []
    direction = -1
    row, col = 0, 0
    for i in range(len(cipher)):
        if row == 0 or row == rails - 1:
            direction *= -1
        result.append(fence[row][col])
        col += 1
        row += direction
    return "".join(result)

message = "WE ARE DISCOVERED FLEE AT ONCE"
rails = 3

print("Rail Fence Cipher:")
print(f"Original: {message}")
encrypted_rail = rail_fence_encrypt(message.replace(' ', ''), rails)
print(f"Encrypted: {encrypted_rail}")
decrypted_rail = rail_fence_decrypt(encrypted_rail, rails)
print(f"Decrypted: {decrypted_rail}")

```

```

Rail Fence Cipher:
Original: WE ARE DISCOVERED FLEE AT ONCE
Encrypted: WECRLTEERDSOEFEAOCAIVDEN
Decrypted: WEAREDISCOVEREDFLEEATONCE

```

✓ Practical 10

```

def get_permutation_order(key):
    key_tuples = []
    for i, char in enumerate(key):
        key_tuples.append((char, i))
    key_tuples.sort()
    permutation_order = [item[1] for item in key_tuples]
    return permutation_order

def row_transposition_encrypt(plaintext, key):
    plaintext = plaintext.replace(' ', '').upper()
    num_cols = len(key)
    num_rows = (len(plaintext) + num_cols - 1) // num_cols
    padded_len = num_rows * num_cols
    plaintext += 'X' * (padded_len - len(plaintext))
    matrix = [[''] * num_cols for _ in range(num_rows)]
    char_idx = 0
    for r in range(num_rows):

```

```

        for c in range(num_cols):
            matrix[r][c] = plaintext[char_idx]
            char_idx += 1
    enc_order = get_permutation_order(key)
    ciphertext_chars = []
    for col_idx_in_orig_matrix in enc_order:
        for r in range(num_rows):
            ciphertext_chars.append(matrix[r][col_idx_in_orig_matrix])
    return "".join(ciphertext_chars)

def row_transposition_decrypt(ciphertext, key):
    num_cols = len(key)
    num_rows = len(ciphertext) // num_cols
    matrix = [[''] * num_cols for _ in range(num_rows)]
    enc_order = get_permutation_order(key)
    current_char_idx = 0
    for col_idx_in_orig_matrix in enc_order:
        for r in range(num_rows):
            matrix[r][col_idx_in_orig_matrix] = ciphertext[current_char_idx]
            current_char_idx += 1
    plaintext_chars = []
    for r in range(num_rows):
        for c in range(num_cols):
            plaintext_chars.append(matrix[r][c])
    return "".join(plaintext_chars)

message = "WE ARE DISCOVERED FLEE AT ONCE"
cipher_key = "ZEBRAS"

print("Row Transposition Cipher:")
print(f"Original: {message}")
encrypted_row = row_transposition_encrypt(message, cipher_key)
print(f"Encrypted: {encrypted_row}")
decrypted_row = row_transposition_decrypt(encrypted_row, cipher_key)
print(f"Decrypted: {decrypted_row}")

```

```

Row Transposition Cipher:
Original: WE ARE DISCOVERED FLEE AT ONCE
Encrypted: EVLNXACDTXSEAXROFOXDEECXWIREE
Decrypted: WEAREDISCOVEREDFLEEATONCEXXXXX

```

✓ Practical 11

```

def product_cipher_encrypt_transposition(plaintext, caesar_shift, trans_key):
    intermediate_text = caesar_cipher(plaintext, caesar_shift, mode='encrypt')
    encrypted_text = row_transposition_encrypt(intermediate_text, trans_key)
    return encrypted_text

def product_cipher_decrypt_transposition(ciphertext, caesar_shift, trans_key):
    intermediate_text = row_transposition_decrypt(ciphertext, trans_key)
    decrypted_text = caesar_cipher(intermediate_text, caesar_shift, mode='decrypt')
    return decrypted_text

message_prod = "ATTACK AT DAWN"
caesar_shift_prod = 3
transposition_key_prod = "KEY"

```

```
print("Product Cipher (Caesar + Row Transposition):")
print(f"Original: {message_prod}")

encrypted_prod = product_cipher_encrypt_transposition(message_prod, caesar_shift_prod,
print(f"Encrypted: {encrypted_prod}")

decrypted_prod = product_cipher_decrypt_transposition(encrypted_prod, caesar_shift_prod
print(f"Decrypted: {decrypted_prod}")
```

```
Product Cipher (Caesar + Row Transposition):
Original: ATTACK AT DAWN
Encrypted: WFWZDDDDWNGQ
Decrypted: ATTACKATDAWN
```